

# Model-Based Approach for Checking Consistency Between Specifications and Code Through Generative AI

Reo Kawaguchi  
DENSO CREATE INC  
Aichi, Japan  
reo.kawaguchi.j4n@jpgr.denso.com

Kaori Hayashi  
DENSO CREATE INC  
Aichi, Japan  
kaori.hayashi.j4e@jpgr.denso.com

Hiroyuki Sato  
DENSO CREATE INC  
Aichi, Japan  
hiroyuki.sato.j3g@jpgr.denso.com

Shinnosuke Fuwa  
DENSO CREATE INC  
Aichi, Japan  
shinnosuke.fuwa.j8h@jpgr.denso.com

Nobuhide Kobayashi  
DENSO CREATE INC  
Aichi, Japan  
nobuhide.kobayashi.j5m@jpgr.denso.com

Hironori Takeuchi  
Musashi University  
Tokyo, Japan  
h.takeuchi@cc.musashi.ac.jp

**Abstract**—The use of generative AI in software development is expanding across almost all development phases, including requirements analysis, design, implementation, and testing. In some cases, software is developed using generative AI throughout the entire development process. In this context, quality assurance of software artifacts generated by generative AI has become increasingly important because the generation process is often a black box. In this study, we define consistency checking as verifying that the contents of a specification are correctly and completely reflected in the corresponding code without omissions or errors, and investigate a method for performing such checking. We propose a metamodel for representing the contents of specifications and code as models, and a method for automating consistency checking using generative AI based on the proposed metamodel. We implemented the proposed method and applied it to artifacts from a business software system used in practice, consisting of a 76-page specification and approximately 42,000 lines of code. The results showed that the True Positive Rate (TPR), representing the proportion of inconsistencies correctly detected, was 92.6%, while the False Positive Rate (FPR), representing the proportion of consistent items incorrectly detected as inconsistent, was 5.5%. Furthermore, we found that a practitioner was able to perform the final review of the resulting consistency-checking results within eight hours, demonstrating that the proposed method can be effectively applied to actual software development projects.

**Index Terms**—consistency checking, specification, code, generative AI, model-based verification

## I. INTRODUCTION

The integration of artificial intelligence (AI) technologies into software development has a long history. Natural language processing (NLP) and machine learning (ML), two core pillars of AI, have been explored as a means of automating and supporting various software engineering tasks for decades [1]. Early efforts focused on tasks such as code search, bug detection, and document classification, but their practical impact was limited by the capabilities of the models available at the time. The landscape shifted significantly in the 2010s

with the rapid advancement of deep learning, which brought substantial improvements in the performance of NLP and ML models. More recently, the emergence of generative AI has begun to fundamentally transform software development practices. In particular, large language models (LLMs) are now being actively explored across all development phases, such as requirements, design, implementation, and testing [2].

In recent years, the importance of software in automobiles has been growing, and the scale of automotive software development has been expanding year by year. As development scales up, the adoption of generative AI is expected to accelerate as a means of improving development efficiency.

In traditional software quality assurance, the fundamental approach has been to verify the validity of deliverables based on an understanding of the content of design documents and source code, as well as the intent of the developers. Generative AI, however, produces deliverables through a process that is essentially a black box, leaving the intent behind the design and code unclear. As a result, quality assurance in generative AI-assisted development introduces additional costs not required in conventional development processes, and presents the challenge that the accuracy of quality verification depends on the experience and skill level of the personnel involved. Nevertheless, methods for supporting quality assurance in generative AI-assisted development have not yet been sufficiently established.

In this paper, we address the challenges described above by focusing on verifying that the content of specifications is reflected in the code accurately, completely, and without error, and we propose a method for checking consistency between specifications and code. Since specifications and code handle information at different levels of granularity, we define a metamodel that enables comparison of specifications and code on a common basis, making consistency checking possible. Based on the defined metamodel, specifications and code are

each modeled, and consistency checking is then carried out according to the proposed method. Through this approach, we aim to reduce the cost required to understand and review deliverables, and to improve the accuracy of consistency checking.

The remainder of this paper is organized as follows. Section II provides an overview of related work. Section III then describes the subject of this study and the research issues addressed. Section IV presents the proposed method, while Section V evaluates its effectiveness through experiments using artifacts from an actual development project. Section VI discusses the effects and limitations of the proposed method, and Section VII concludes the paper.

## II. RELATED WORK

In this study, we investigate a method for checking the consistency between software development artifacts by modeling these artifacts. We also investigate the use of generative AI to automate this process. This section reviews existing studies related to these two aspects.

In software development, modeling is widely used to facilitate an understanding of the system being developed. UML is one of the most widely used approaches for providing a unified notation for software models [3]. In addition to UML models, structural models for describing requirements [4], [5] and models specialized for specific requirements analysis tasks [6] have been proposed to facilitate a shared understanding of the system and its requirements. There have also been studies on generating UML-based models from specifications written in natural language [7], as well as on generating natural-language specifications from UML-based models [8]. Furthermore, studies have investigated approaches to supporting the understanding of the large amount of code produced during software development. For example, methods have been proposed to visualize relationships among code elements using models [9]. Tools for visualizing the quality of developed code have also been developed [10], [11].

Meanwhile, the use of LLMs in software development has been rapidly expanding. At the requirements stage, efforts have been made not only to automate individual activities such as requirements elicitation [12], model generation for requirements analysis [13], and artifact quality analysis [14], but also to automate the overall development process [15]. At the implementation stage, practices involving the use of LLMs within organizations have been reported [16], while approaches such as automated unit test generation have been proposed for the testing stage [17]. In addition, practices have been reported in which engineers use LLMs throughout the development process, from user requirements to implementation and testing, in an end-to-end manner [18]. At the same time, the impact of generative AI services on software development has been investigated [19]–[21]. These studies assume development settings in which both generative AI services and human developers contribute to the creation of software artifacts, and discuss development approaches and

challenges based on the respective characteristics of generative AI and human developers.

As described above, various methods have been proposed for analyzing specifications and code individually, as well as for automating such analysis activities. However, relatively little attention has been paid to examining the relationships and consistency between the contents of these artifacts. As pointed out in [21], as the use of generative AI in software development continues to expand, engineers who work closely with end users may also be required to take responsibility for quality assurance, including consistency checking between such artifacts. Therefore, support for these quality assurance activities is needed.

## III. RESEARCH SUBJECT AND RESEARCH QUESTION

### A. *Quality Attributes for Evaluating the Quality of Software Development Artifacts*

In software development projects, the quality of specifications and code is evaluated to ensure the quality of the software being developed. For example, the quality of artifacts is analyzed according to the quality perspectives [23]. IEEE 29148 [22] is a standard that defines such quality perspectives, and natural language processing techniques have also been used to analyze the quality of specification descriptions from these perspectives [24].

In this study, we investigate whether the quality incorporated into a specification is ultimately reflected in the resulting software. Therefore, we focus on the quality characteristics of software products defined in ISO/IEC 25010:2023 [25]. The standard defines the following nine quality characteristics as the Product Quality Model for software.

- **Functional Suitability**

- The system provides the required functions accurately and completely.

- **Performance Efficiency**

- The system satisfies performance requirements, such as response time, throughput, and resource utilization.

- **Compatibility**

- The system can coexist and interoperate with other systems.

- **Interaction Capability**

- Users can easily understand, learn, and operate the system.
- The system enables smooth and efficient interaction with users.

- **Reliability**

- The system operates stably under expected conditions of use and even when failures occur.

- **Security**

- The system maintains confidentiality, integrity, and availability, and prevents unauthorized access and tampering.

- **Maintainability**

- The system can be easily modified, maintained, extended, and tested.
- **Flexibility**
  - The system can be easily adapted, extended, and re-configured in response to changes in its environment and requirements.
- **Safety**
  - System failures or malfunctions do not cause harm to human life, property, or the environment.

Based on these quality characteristics, we investigate a method for checking whether the quality incorporated into the specification of the software being developed is reflected in the code that implements it.

#### B. Consistency Checking Between Artifacts and Research Question

In this study, we consider a situation in which not only human application developers but also generative AI generates code from a specification and investigate how to check consistency between the specification and the code. The specification describes the functions of the software to be developed as well as the quality requirements that the software should satisfy. To determine whether the developed software satisfies these quality requirements, dynamic testing using test cases is performed, along with static testing such as manual reviews of individual artifacts. Methods for using generative AI to effectively support manual reviews by developers have also been investigated [14], [26], [27].

On the other hand, verifying that the contents described in a specification are correctly reflected in the code requires extensive comparative analysis between the artifacts. Studies such as [28], [29] have used generative AI to effectively perform such consistency checking between artifacts, investigating consistency between requirements descriptions. However, consistency checking between specifications and code has not yet been sufficiently investigated. In software development, specifications and code differ in their representations and levels of granularity, making direct comparison difficult. Consequently, consistency checking between the two is often a manual task that depends on the experience and skills of individual practitioners.

In particular, with the increasing use of generative AI, there may be situations in which practitioners are required to check consistency without fully understanding the content or intent of the generated code. In such situations, it becomes increasingly important to perform consistency checking between specifications and code objectively with the support of generative AI.

Therefore, this study addresses the following research question (RQ):

**RQ** Can consistency between specifications and code be effectively checked using generative AI?

To address this RQ, we propose a method that transforms specifications and code into comparable models according to

the quality characteristics targeted by the consistency checking. The method clarifies the correspondence between the two artifacts and enables their consistency to be checked. We then evaluate the effectiveness of the proposed method using artifacts produced in an actual software development project.

## IV. PROPOSAL

### A. Overview

In this study, we propose a method for automatically checking the consistency between design specification documents and source code using generative AI. The proposed method takes design specification documents and source code as inputs and identifies inconsistencies between them.

Figure 1 presents an overview of the proposed method, which consists of three components:

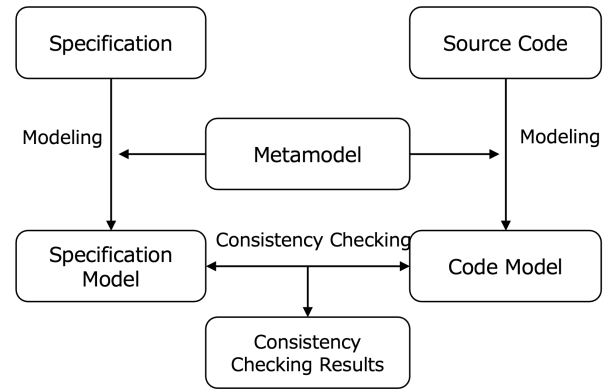


Fig. 1. Overview

- 1) Metamodel
- 2) Modeling
- 3) Consistency checking

First, the specification documents and source code are transformed into corresponding models based on the meta-model and the respective modeling methods. Subsequently, the consistency between the two models is checked from the defined assessment viewpoints.

### B. Metamodel

In this study, we model specifications and code according to each quality characteristic. Among the Product Quality characteristics described in the previous section, we focus on Functional Suitability. In addition, we introduce Privacy as another quality characteristic for consistency checking. For each characteristic, we define the following assessment viewpoints.

- 1) Functional Suitability
  - a) Each data item is handled as specified in the requirements.
  - b) Each file is handled as specified in the requirements.

- c) The inputs and outputs of each function and the inputs and outputs to and from external applications conform to the requirements.
- d) The data initialized at the initial startup and their initial values conform to the requirements.
- e) View transitions conform to the requirements.
- f) The static UI requirements are satisfied.

2) Privacy

- a) Personal data is handled as specified in the requirements.

For viewpoints 1-a, 1-b, 1-d, 1-f, and 2-a, consistency can be checked by comparing how the items to be verified are described in the specification and code, respectively. Therefore, for these viewpoints, we extract the relevant descriptions from the specification and code for each item and represent them as tabular models. The items of interest for each viewpoint constitute the metamodel of the corresponding tabular model, as shown in Table I.

TABLE I  
METAMODEL OF THE TABULAR MODEL

| Viewpoint | Items to be checked (Table Items)                                                                                                                                                |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-a       | Data type, value range, and data lifetime                                                                                                                                        |
| 1-b       | File name, file extension, input/output file path, purpose of the JSON file, field name, type, constraints, and additional rules                                                 |
| 1-d       | Initial value of data                                                                                                                                                            |
| 1-f       | Data source of each screen/component, binding information, type of UI element of each screen/component, display content of each screen/component, placeholder, and setting value |
| 2-a       | Method for protecting personal data                                                                                                                                              |

On the other hand, for viewpoints 1-c and 1-e, we use graphical models to represent the specification and code. For viewpoint 1-c, we first model the functions. EARS [4], [5] is a method for modeling functions. In this study, based on an extension of EARS [6], we define a function model consisting of the inputs and outputs of a function and the operational trigger that initiates the function. Events observed in relation to the operational triggers and inputs are also represented as elements of the model. Figure 2 shows the metamodel for functions.

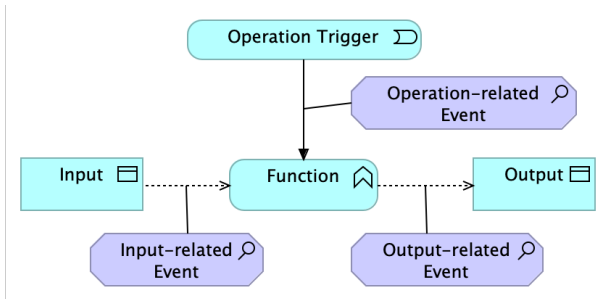


Fig. 2. Function Metamodel

For viewpoint 1-e, we define a state transition diagram consisting of the following elements as the metamodel:

- Flow of view transitions
- Conditions for view transitions

### C. Modeling Methods

For viewpoints 1-a, 1-b, 1-d, 1-f, and 2-a, which use tabular models, generative AI extracts the items defined in the meta-model from the specification and creates the corresponding tabular models. From the code, generative AI creates tabular models for each viewpoint based on the following instructions.

- 1-a Extract data used for input/output with external entities and for UI display, identify the type of each data item, and record them in a table. For the value range, identify the specific values used for each data item in the code and record them in the table. For the lifetime, if the lifetime of a data item is represented in the code, identify it and record it in the table.
- 1-b Identify file input/output operations used in the code and record the information required to ensure the quality of the corresponding files in a table. For JSON files, extract information such as the field names, types, and constraints used in the files and record them in the table.
- 1-d Examine setting files and other relevant files and record the data types and initial values in a table.
- 1-f Extract the specified information from the code that defines each view and record it in a table.
- 2-a Identify the lifetime of variables that store or process personal data and record the protection mechanisms applied to the data in a table.

Next, we describe the construction of the graphical model for viewpoint 1-c. The modeling procedure is shown in Figure 3.

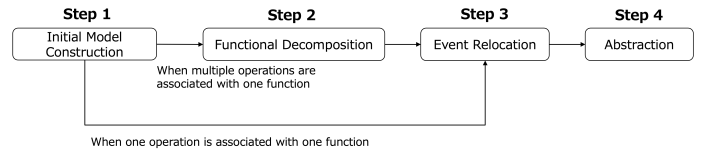


Fig. 3. Overview of the Modeling Approach for Viewpoint 1-c

The procedure consists of four steps. In Step 1, an initial model is created for each function based on the metamodel shown in Figure 2. For the specification, the model is created by generative AI according to the following procedure:

- 1) Extract the input/output data, operations, and events related to each function from the specification.
- 2) Based on the metamodel, associate the input/output data and operations with each function.
- 3) Among the input/output data, add models for data exchanged with views or external applications and associate them with the corresponding data items. Data that are not exchanged with views or external applications, i.e., data used only for internal processing within a function, are excluded from the model.

- 4) Identify the elements to which the events obtained from the descriptions in the specification should be associated.
  - Identify the data or operation modified by each extracted event.
  - Associate the event with the relationship between the function and the corresponding data or operation.

For the code, data, methods, events, operations, external applications, and views related to each function are extracted. For each method, the data flow is modeled based on the metamodel. Data that are not exchanged with external applications or views are considered to be used for internal processing and are therefore excluded from the model and treated as data internal to the function. Data exchanged with external applications or views are treated as input/output data of the function and associated with the corresponding views or external applications. In Step 2, functions are divided so that each function has a one-to-one correspondence with its triggering operation. If a one-to-one correspondence between functions and operations has already been established in Step 1, Step 2 is skipped and the procedure proceeds to Step 3. There are two reasons for dividing functions:

- The output data corresponding to a single triggering operation cannot be uniquely determined.
- The temporal order of operations cannot be correctly represented in the model.

To address these issues, functions are divided according to the following procedure:

- 1) Divide the functions so that exactly one operation is associated with each function.
- 2) Associate the output data related to each operation with the corresponding function.

For the specification model, the placement of events in the initial model created in Step 1 depends on how they are expressed in the specification. For example, consider the statement, “In the judgment function, when the OK button is pressed outside the operating hours, a warning message is sent.” When the modifying relationships in this statement are analyzed, the event “outside the operating hours” is associated with the operation “pressing the OK button.” However, this expression means that “a warning message is sent outside the operating hours.” Therefore, to enable comparison with the code, the event needs to be associated with “sending a warning message.” Accordingly, in Step 3, the position of each event is adjusted according to the following procedure:

- 1) Identify the condition under which the event occurs.
- 2) Associate the event with the target data specified under that condition.

Step 3 is skipped for the code model. Finally, in Step 4, the levels of granularity of the specification model and code model are aligned. Specifically, output data and events that are

divided according to different values are grouped into a single element.

Finally, we describe the construction of the graphical model for viewpoint 1-e. When constructing the specification-side model, if a view transition diagram is included in the specification, it is used directly as the specification-side model. If no view transition diagram is provided, the transition relationships between each view and the starting point are represented using arrows. When transition conditions are explicitly specified, the conditions are described on the corresponding arrows. For the code-side model, each view and the starting point are modeled, and transition relationships are represented using arrows based on the transition processing implemented in the code. When transition conditions exist, they are described on the corresponding arrows.

#### *D. Consistency Checking*

The specification models and code models created for each viewpoint are compared using generative AI to check their consistency. When comparing the specification and code models, whether different notations, such as variable names, should be treated as referring to the same entity is left to the reasoning capability of generative AI.

For viewpoints 1-a, 1-b, 1-d, 1-f, and 2-a, we define the following checking procedures for the created tabular models and instruct generative AI to perform consistency checking.

- 1-a Compare the tables created from the specification and code. Verify that the data types, value ranges, and lifetimes are consistent.
- 1-b Compare the tables created from the specification and code. Verify that the file information is consistent.
- 1-d Check the initial values set for each data type. Verify that the initial values are consistent between the specification and code.
- 1-f Check the data source and binding information for each screen/component. Check the type of UI element and its displayed content. Check the placeholders and setting values. Verify that the controls placed on each screen/component are correct.
- 2-a Verify that the methods for protecting personal data are consistent between the specification and code.

For viewpoint 1-c, we define the following checking procedures for the created graphical models and use generative AI to perform consistency checking.

- 1) Verify that the associations between functions and operations are correct.
- 2) Verify that the number and contents of the input/output data of each function are consistent. If the number or contents differ, the difference may be caused by different levels of granularity in the descriptions of the specification and code. In such cases, generative AI determines whether the data in the code model can be combined to correspond to the data in the specification model, and thereby assesses their consistency.

- 3) Verify that the contents of the events and their associated positions are consistent.

For viewpoint 1-e, consistency is checked by verifying that the flow of view transitions and the transition conditions are identical.

### E. Example of Modeling and Consistency Checking

Here, we present an example of applying the modeling and consistency-checking methods described in the previous sections. The target is a GUI application running on a PC called the “Overtime Management Tool.” An overview of the GUI application is described in the Appendix (Figure 7). Among the results obtained by applying the proposed method to the specification and code of this application, the models and consistency-checking results for viewpoints 1-a and 1-c are presented below.

For viewpoint 1-a, the specification model and code model are shown in Tables II and III, respectively. Although the data names in the specification are written in Japanese, they are translated into English in Table II. The consistency-checking results using these models identified inconsistencies in which the value ranges of the data items “Actual Working Time,” “Login ID,” and “Scheduled Overtime,” as well as the lifetimes of “Login ID” and “Password,” in the specification were not correctly reflected in the code.

TABLE II  
EXAMPLE OF SPECIFICATION MODEL FOR VIEWPOINT 1-A

| Data Name              | Type<br>(in External Tool) | Type   | Value Range                                                                         | Lifetime                       |
|------------------------|----------------------------|--------|-------------------------------------------------------------------------------------|--------------------------------|
| Actual Working Time    | ActualTimeEntry            | float  | 0--3,825                                                                            | None                           |
| Login ID               | string                     | string | Windows user name setting rules                                                     | Data is not retained after use |
| Password               | None                       | string | 0--32,767 characters                                                                | Data is not retained after use |
| Employment Status      | None                       | string | "Work", "Paid Leave", "AM Leave", "PM Leave", "Holiday"                             | None                           |
| Scheduled Working Time | None                       | string | Hour: [0--12] in 1-hour increments; Minute: [0, 25, 50, 75] in 15-minute increments | None                           |
| Scheduled Overtime     | None                       | string | - 40 -- 48.75                                                                       | None                           |

TABLE III  
EXAMPLE OF CODE MODEL FOR VIEWPOINT 1-A

| Data Name                                                                 | Type<br>(In External Tool) | Type   | Value Range                                                                                  | Lifetime |
|---------------------------------------------------------------------------|----------------------------|--------|----------------------------------------------------------------------------------------------|----------|
| worktime                                                                  | ActualTimeEntry            | float  | Range of float values                                                                        | None     |
| LoginId                                                                   | string                     | string | Range of string values                                                                       | None     |
| Password                                                                  | None                       | string | 0--32,767 characters                                                                         | None     |
| Kmon, Ktue,<br>Kwed, Kthuh,<br>Kfri                                       | None                       | string | "Work", "Paid Leave", "AM<br>Leave", "PM Leave", "Holiday"                                   | None     |
| Hmon, Mmon,<br>Htue, Mtue,<br>Hwed, Mwed,<br>Hthuh, Mthuh,<br>Hfri, Mfri, | None                       | string | Hour: [0--12] in 1-hour<br>increments<br>Minute: [0, 25, 50, 75] in 15-<br>minute increments | None     |
| planWT                                                                    | None                       | string | Range of string values                                                                       | None     |

Next, for viewpoint 1-c, the specification model and code model are shown in Figures 4 and 5, respectively. The model

elements extracted from the specification and processing descriptions are presented in Japanese. The results of analyzing

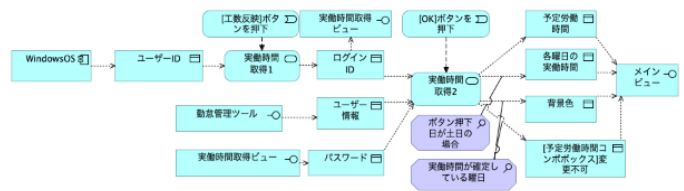


Fig. 4. Example of Specification Model for Viewpoint 1-c

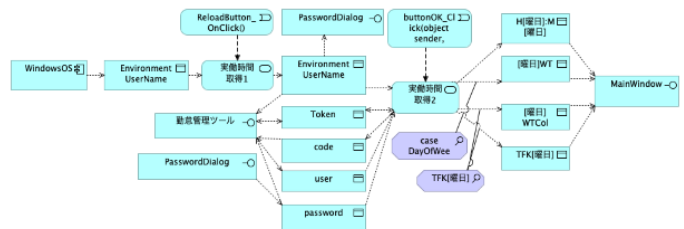


Fig. 5. Example of Code Model for Viewpoint 1-c

the three consistency-checking items for viewpoint 1-c using these models are as follows:

- 1) The associations between functions and operations are consistent between the specification model and code model.
- 2) The input and output data are consistent for the function “実働時間取得 1.”<sup>1</sup>
- 3) The number of events and their associated elements are consistent.

For “実働時間取得 2,” the specification model contains “User Information,” whereas the code model contains “Token,” “code,” and “user.” Generative AI determines that this difference is attributable to a difference in the level of granularity. Therefore, the consistency-checking result for viewpoint 1-c indicates that the specification and code are consistent.

## V. EVALUATION EXPERIMENT

In this section, we evaluate the effectiveness of the proposed method by applying it to an actual design specification document and source code. Figure 6 shows an overview of the system developed for the evaluation experiment.

The evaluation system takes a design specification document and source code as inputs, automatically performs consistency checking, and outputs the results. The system consists of three agents, Agent 1–3, which use generative AI. Following the proposed method, Agent 1 and Agent 2 generate a model from the design specification document and source code, respectively, while Agent 3 performs consistency checking based on the generated models. The three agents operate independently and are implemented as Custom Agents using

<sup>1</sup>実働時間取得: Retrieve Actual Working Time

TABLE IV  
EVALUATION RESULTS

| Viewpoint | Judgments | Detected Inconsistencies | TP | FP | FN | TN   | TPR    | FPR  |
|-----------|-----------|--------------------------|----|----|----|------|--------|------|
| 1-a       | 90        | 0                        | 0  | 0  | 0  | 90   | –      | 0.0% |
| 1-b       | 115       | 3                        | 3  | 0  | 0  | 112  | 100.0% | 0.0% |
| 1-c       | 5         | 0                        | 0  | 0  | 0  | 5    | –      | 0.0% |
| 1-d       | 0         | 0                        | 0  | 0  | 0  | 0    | –      | –    |
| 1-e       | 172       | 22                       | 16 | 6  | 0  | 150  | 100.0% | 3.8% |
| 1-f       | 888       | 68                       | 6  | 62 | 2  | 818  | 75.0%  | 7.0% |
| 2-a       | 2         | 0                        | 0  | 0  | 0  | 2    | –      | 0.0% |
| Total     | 1272      | 93                       | 25 | 68 | 2  | 1177 | 92.6%  | 5.5% |

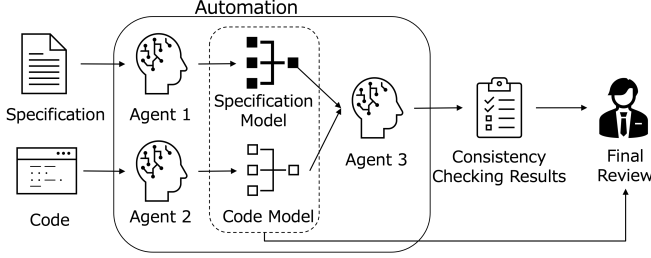


Fig. 6. Evaluation System

the GitHub Copilot extension for Visual Studio Code. The language model used by the agents was GPT-5.1, and a separate set of Agents 1–3 was implemented for each consistency-checking viewpoint. This separation of the modeling and consistency checking processes allows the evaluation to follow the structure of the proposed method.

In the evaluation experiment, we used an existing software system that had been developed manually, together with its design specification document and source code. The target software is a desktop GUI application for office use, of the same type as the example presented in Section IV-E, but developed for actual business use at a larger and more practical scale. The design specification document consists of 76 pages, and the source code contains approximately 42,000 lines. The results of the consistency checking, together with the models used for the checking, were subsequently reviewed by a practitioner. The practitioner who conducted the final review had one year of software development experience and had not been involved in the development of the target software. Thus, the final review was conducted by a person who was not familiar with the development history of the target software. This setting was intended to evaluate the consistency checking results independently of the developers who originally created the artifacts.

Since the objective of consistency checking is to detect inconsistencies accurately while minimizing both false negatives and false positives, the results produced by the system were classified as follows during the final review:

- TP (True Positive): An actual inconsistency is correctly identified as an inconsistency.

- FP (False Positive): A case that is actually consistent is incorrectly identified as an inconsistency.
- TN (True Negative): A case that is actually consistent is correctly identified as consistent.
- FN (False Negative): An actual inconsistency is incorrectly identified as consistent.

As with anomaly detection, an important objective of consistency checking is to achieve a high rate of correctly detecting inconsistencies and to avoid missing inconsistencies. Therefore, we use the following True Positive Rate (TPR) as an evaluation metric:

$$TPR = \frac{TP}{TP + FN} \quad (1)$$

At the same time, reducing the rate of incorrectly identifying consistent cases as inconsistent is important for reducing the effort required to review the checking results. For this purpose, we use the following False Positive Rate (FPR) as an evaluation metric:

$$FPR = \frac{FP}{TN + FP} \quad (2)$$

Table IV presents the classification results and the values of the evaluation metrics (TPR and FPR) for each viewpoint. A dash (–) indicates that the corresponding metric could not be calculated. In this experiment each judgment is further evaluated through the final review, which determines whether each detected inconsistency is actually supported by the specification and source code. For example, in the case of Viewpoint 1-e, 172 items were checked using the constructed models, of which 22 were identified as inconsistent by the consistency checking process. After the results were reviewed by a practitioner, the TPR was calculated as  $16/(16 + 0) = 100\%$ , while the FPR was calculated as  $6/(150 + 6) = 3.8\%$ .

The overall evaluation results show that the TPR was 92.6%, with only two inconsistencies missed between the design specification document and the source code. The FPR was 5.5%, and 1,177 of the 1,245 cases that were actually consistent between the design specification document and the source code were correctly identified as consistent. These results indicate that the proposed method can perform consistency checking between design specification documents and source code with a practically acceptable level of accuracy. The final review by

the practitioner required eight hours. Considering the size of the artifacts used in the experiment, this result indicates that the proposed method can support the review of consistency checking results within a limited amount of human effort. These results indicate that the proposed method provides an effective answer to our RQ: generative AI can be used to check consistency between specifications and code with a practically acceptable level of accuracy, and the proposed method can be applied effectively in actual software development projects.

On the other hand, 68 cases were incorrectly identified as inconsistent even though the design specification document and source code were actually consistent (FP). Reducing the number of FPs is therefore necessary to further reduce the FPR. We analyzed the causes of the FPs according to the components of the proposed system shown in Figure 6. The breakdown of the causes was as follows:

- Description in the design specification document: 52 cases (76.5%)
- Description in the source code: 0 cases (0.0%)
- Generation of the specification model: 2 cases (2.9%)
- Generation of the code model: 14 cases (20.6%)
- Consistency checking between models: 0 cases (0.0%)

Approximately 79% of the FPs were caused by the way the design specification document was described. In particular, a large number of FPs occurred in parts where the UI specifications were described only using images. Depending on the resolution of images included in the input document, the generative AI used in the experiment may not have been able to sufficiently interpret the information contained in the images. This is considered to have resulted in omissions or errors in extracting model elements. Therefore, the quality of the input specification document can affect the quality of the generated models and, consequently, the consistency checking results.

Therefore, supplementing image-based UI specifications with textual descriptions may make the specification document easier for the AI to interpret and is expected to reduce the number of FPs. No incorrect judgments attributable to the consistency checking between the models themselves were observed. These results suggest that, when model elements are appropriately extracted from the design specification document and source code, consistency checking can be performed with high accuracy.

## VI. DISCUSSION

### A. Effectiveness of the Proposed Method

These findings, discussed in detail below, address our RQ on whether consistency between specifications and code can be effectively checked using generative AI. The proposed method models specification documents and source code based on metamodels defined for each viewpoint and then performs consistency checking between the resulting models. The evaluation results demonstrated that the proposed method can detect inconsistencies with sufficient accuracy even for

artifacts of a scale intended for business use. Furthermore, the fact that an evaluator with only one year of development experience was able to identify actual inconsistencies suggests that the proposed method can perform consistency checking without relying heavily on specialized knowledge of the target software. This is considered to be attributable to the fact that the proposed method limits the objects to be checked according to each viewpoint and compares them at a consistent level of granularity. Thus, the proposed method can provide a structured basis for checking consistency between different types of software artifacts.

Furthermore, the proposed method is not limited to establishing traceability between artifacts. Instead, it produces consistency checking results according to the defined viewpoints. This allows practitioners to identify which aspects of the specification and source code are inconsistent, rather than simply determining whether the artifacts are related to each other. This suggests that the proposed method can reduce the subjectivity of the checking process, further supporting its potential for effective use on business-scale artifacts.

Although the evaluation experiment was conducted using specification documents and source code developed by human developers, the proposed method can also be applied to artifacts generated by generative AI. As the use of generative AI in software development continues to advance, the roles of humans and AI in the development process have been actively discussed [20], [30]. As generative AI becomes increasingly capable of producing various software artifacts, human involvement in verifying the quality of these artifacts may become a bottleneck in the development process. In such a situation, the proposed method is expected to provide an effective approach for performing consistency checking between software artifacts.

### B. Threats to Validity

The evaluation experiment targeted a GUI application running on a PC. Therefore, the effectiveness of the proposed method for other types of software, such as Web applications and embedded software, has not yet been confirmed. In addition, it has not been confirmed whether the viewpoints used for consistency checking are sufficiently comprehensive across different application domains. The robustness of the consistency checking results against changes in the generative AI model has also not been evaluated. Further validation is therefore necessary by applying the proposed method to a wider variety of software development artifacts using different generative AI models.

In this study, specification documents and source code were represented using models that could be compared with each other, and consistency checking was performed based on these models. However, design specification documents may also specify architectural design patterns to be followed in implementing particular functions. For example, a specification document may state an implementation policy such as “implement the user management function according to the Role-Based Access Control (RBAC) pattern.” In such a

case, it is necessary to verify whether the implemented code conforms to the RBAC pattern. The proposed method cannot currently perform consistency checking to determine whether such implementation policies are correctly reflected in the source code. Addressing this issue will require extending the proposed method, including investigating model representations for information such as implementation policies and approaches.

In addition, this study does not address the severity of inconsistencies identified by the proposed method or how they should be resolved. Investigating how to effectively utilize the generated consistency checking results will be important for practical application of the proposed method. These are our future work.

## VII. CONCLUSION

The use of generative AI is rapidly advancing across almost all phases of software development, including requirements analysis, design, implementation, and testing. Development processes that rely on generative AI from beginning to end are also being explored. As software artifacts are increasingly produced not only by humans but also by generative AI, ensuring the quality of these artifacts has become an important concern.

In this study, we focused on quality assurance of software artifacts and defined consistency checking as verifying that the contents of a specification document are correctly reflected in the source code without omissions, additions, or errors. We then proposed a method for performing consistency checking between software artifacts using generative AI. Specifically, we defined quality viewpoints, developed a metamodel for each viewpoint, and represented the contents of specification documents and source code as models based on the corresponding metamodels. We then performed consistency checking by comparing the resulting models. This approach enables consistency to be checked from specific quality viewpoints while reducing the differences in representation and granularity between the two artifacts.

We built a system that implements the proposed method using generative AI and applied it to artifacts from an actual software development project. In the application to software artifacts consisting of a 76-page specification document and approximately 42,000 lines of source code, the True Positive Rate (TPR), which represents the proportion of inconsistencies correctly detected, was 92.6%, while the False Positive Rate (FPR), which represents the proportion of consistent cases incorrectly detected as inconsistent, was 5.5%. These results demonstrated that the proposed method can be effectively applied in an actual software development environment. The results also indicate that the proposed method can support consistency checking for relatively large-scale software artifacts.

Future work includes validating the applicability of the proposed method to a wider variety of software development artifacts through practical applications, as well as extending the method to cover information that is difficult to represent as models, such as implementation policies.

## REFERENCES

- [1] A. Mastropaolo, C. Escobar-Velázquez, and M. Linages-Vásquez, "From triumph to uncertainty: The journey of software engineering in the AI Era," *ACM Transaction on Software Engineering and Methodology*, vol. 34, no. 5, pp. 131:1 – 231:34, 2025.
- [2] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou, "Large language model-based agents for software engineering: A survey," *ACM Transactions on Software Engineering and Methodology*, 2026.
- [3] G. Booch, J. Rumbaugh, and I. Jacobson, *Unified Modeling Language User Guide*. Addison-Wesley Professional, 2005.
- [4] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, "Easy approach to requirements syntax," in *Proceedings of the 17th IEEE International Requirements Engineering Conference*, 2009, pp. 317 – 322.
- [5] A. Mavin and P. Wilkinson, "Ten Years of EARS," *IEEE Software*, vol. 36, no. 5, pp. 10–14, 2019.
- [6] M. Hayashi, K. Hayashi, Y. Sonobe, H. Sato, and H. Takeuchi, "Specification description analysis method for requirement modeling," in *Proceedings of the 28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (Procedia Computer Science vol. 246)*, 2024, pp. 189 – 198.
- [7] S. Yang and H. Sahraoui, "Towards automatically extracting UML class diagrams from natural language specifications," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 2022, pp. 396–403.
- [8] F. Meziane, N. Athanasakis, and S. Ananiadou, "Generating natural language specifications from UML class diagrams," *Requirements Engineering*, vol. 13, no. 1, pp. 1–18, 2008.
- [9] M. Pinzger, K. Gräfenhain, P. Knab, and H. Gall, "A tool for visual understanding of source code dependencies," in *Proceedings of the 16th IEEE International Conference on Program Comprehension*, 2008, pp. 254–259.
- [10] S. Jin, M. Zhang, Y. Guo, Y. He, Z. Li, B. Chen, B. Zhu, and Y. Xia, "Software code quality measurement: Implications from metric distributions," in *Proceedings of the 23rd IEEE International Conference on Software Quality, Reliability, and Security*, 2023, pp. 488–496.
- [11] F. Hashmat, Z. A. Aljaali, M. Shen, and A. Machiry, "Insights from running 24 static analysis tools on open source software repositories," in *Proceedings of the 20th International Conference Information Systems Security*, 2024, pp. 225–245.
- [12] A. Korn, S. Gorsch, and A. Vogelsang, "LLMREI: Automating requirements elicitation interviews with llms," in *Proceedings of the 33rd IEEE International Requirements Engineering Conference*, 2025, pp. 19 – 30.
- [13] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative ai in modeling tasks: an experience report with chatgpt and uml," *Software and Systems Modeling*, vol. 22, pp. 781–793, 2023.
- [14] S. Lubos, A. Felfernig, T. N. T. Tran, D. Garber, M. E. Mansi, S. P. Erdeniz, and V.-M. Le, "Leveraging LLMs for the Quality Assurance of Software Requirements," in *Proceedings of the 32nd IEEE International Requirements Engineering Conference*, 2024, pp. 389–397.
- [15] M. Ataei, H. Cheong, D. Grandi, Y. Wang, N. Morris, and A. Tessier, "Elictron: An LLM agent-based simulation framework for design requirements elicitation," *Journal of Computing and Software Science in Engineering*, vol. 25, no. 2, pp. 1–12, 2025.
- [16] S. Chatterjee, C. L. Liu, G. Rowland, and T. Hogarth, "The impact of AI tool on engineering at ANZ bank an empirical study on GitHub copilot within corporate environment," *Computer Science & Information Technology*, vol. 14, no. 7, pp. 15–30, 2024.
- [17] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A framework for LLM-Based test generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundation of Software Engineering*, 2024, pp. 572 – 576.
- [18] B. Wei, "Requirements are all you need: From requirements to code with LLMs," in *Proceedings of IEEE 32nd International Requirements Engineering Conference*, 2024, pp. 416 – 422.
- [19] A. E. Hassan, D. Lin, G. K. Rajbahadur, K. Gallaba, F. R. Cogo, B. Chen, H. Zhang, K. Thangarajah, G. Oliva, J. Lin, W. M. Abdullah, and Z. M. Jiang, "Rethinking software engineering in the era of foundation models: A curated catalogue of challenges in the development of trustworthy FMware," in *Companion Proceedings of the 32nd ACM International Conference on the Foundation of Software Engineering*, 2024, pp. 294 – 305.

### Actual Working Time Retrieval

- Enter the login ID and password for TimeTracker NX (a time management tool) through the UI.
- After logging in, retrieve the actual working time for each day of the current week and display them on the UI.
- Retrieve the PC user's ID and automatically enter it as the login ID.
- If the date for which actual working hours are to be retrieved is a Saturday or Sunday, do not retrieve the actual working hours.

### Employment Classification Setting

- Select the employment classification for each day from "Work," "Paid Leave," "Morning Leave," "Afternoon Leave," and "Holiday" through the UI.
- Change the scheduled working time to the standard working time according to the selected work schedule classification.
- Do not change the scheduled working time for days for which the actual working time have been finalized.
- If the employment classification is set to "Paid Leave" or "Holiday," fix the scheduled working hours at "0.00h" and make them unchangeable.
- Display the text color of the employment classification and scheduled working hours combo boxes in gray.
- At the first startup, set the employment classification for all days to "Work."
- At the first startup, display the text in the scheduled working hours combo box in black.
- At the second and subsequent startups, retain the employment classification and scheduled working hours settings from the previous session.

### Scheduled Working Time Setting

- Set the scheduled working time through the UI.
- Hours can be set from "0" to "12" in one-hour increments.
- Minutes can be set to "0," "25," "50," or "75" in 15-minute increments.
- At the first startup, display all scheduled working time as "8.00h."
- At the first startup, allow the scheduled working time to be changed.
- At the second and subsequent startups, retain the settings from the previous session.

### Scheduled Overtime Hours Display

- Display the scheduled overtime hours calculated from the scheduled working hours and work schedule classification on the UI.
- The scheduled overtime hours are calculated using the following formula.  
(Scheduled Overtime Time) = (Total Scheduled Working Time) - (Total Standard Working Hours Based on the Employment Classification)
- When the scheduled working time or employment classification is changed, recalculate the scheduled overtime hours in real time.
- Update the displayed information after recalculation.

Fig. 7. Part of the Specification Document (originally written in Japanese)

- [20] D. Lo, "Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps," in *Proceedings of IEEE/ACM International Conference on Software Engineering: Future of Software Engineering*, 2023, pp. 69 – 85.
- [21] D. Robinson, C. Cabrera, A. D. Gordon, N. D. Lawrence, and L. Mennen, "Requirements are all you need: The final frontier for end-user software engineering," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 141:1–141:22, 2025.
- [22] ISO/IEC/IEEE, *Systems and software engineering — Life cycle processes — Requirements engineering*, Std. ISO/IEC/IEEE 29 148:2018, 2018.
- [23] D. Porter, J. F. DeFranco, and P. Laplante, "Requirements specification automated quality analysis: Past, present, and future," *Computer*, vol. 58, no. 01, pp. 101–104, 2025.
- [24] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, "Natural language processing for requirements engineering: A systematic mapping study," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–41, 2021.
- [25] ISO/IEC, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*, Std. ISO/IEC 25 010:2023, 2023.
- [26] K. C. Pragyan, R. Ghandiparsi, T. Herron, J. Heaps, and M. B. Hosseini, "Demystifying Feature Requests: Leveraging LLMs to Refine Feature Requests in Open-Source Software," in *Proceedings of the 33rd IEEE International Requirements Engineering Conference*, 2025, pp. 104–116.
- [27] S. Santos, T. Breau, T. Norton, S. Haghighi, and S. Ghanavati, "Requirements satisfiability with in-context learning," in *Proceedings of the 32nd IEEE International Requirements Engineering Conference*, 2024, pp. 168–179.
- [28] A. Fantechi, S. Gnesi, L. C. Passaro, and L. Semini, "Inconsistency detection in natural language requirements using ChatGPT: a preliminary evaluation," in *Proceedings of the 31st IEEE International Requirements Engineering Conference*, 2023, pp. 335–340.
- [29] B. Chen, A. A. Babikian, S. Feng, D. Váro, and G. Mussbacher, "LLM-based satisfiability checking of string requirements by consistent data and checker generation," in *Proceedings of the 33rd IEEE International Requirements Engineering Conference*, 2025, pp. 231–243.
- [30] M. Coutinho, L. Marques, A. Santos, M. Dahia, C. França, and R. de Souza Santos, "The role of generative AI in software development productivity: A pilot case study," in *Proceedings of the 1st ACM International Conference on AI-Powered Software*, 2024, p. 131138.

## APPENDIX

This appendix provides an overview of the Overtime Management Tool used in the example application described in Section IV-E. The tool is a GUI application that runs on a PC desktop and is designed to plan working hours on a weekly basis and estimate overtime hours. The specification document for this tool consists of 17 pages. It defines four main functions, and an overview of these functions is shown in Figure 7. The original specification document was written in Japanese.